

Declarative OMP Marketplace Flakes

https://gemini.google.com/app/205d96daa6afd0fa

User prompt: i need an ideas on how to write declarative omp (oh my pi) marketplace flake in order to control its plugins declarative

Response: Declarative control over Oh My Pi (OMP) plugins requires understanding its discovery pipeline: OMP discovers marketplace plugins either via **registry JSON state** (\$XDG_DATA_HOME/omp/plugins/installed_plugins.json and marketplaces.json) or via the **--plugin-dir CLI root injection**.

Three architectural patterns allow you to implement a declarative Nix flake depending on whether you want a global Home Manager setup, an ephemeral project devShell, or a local marketplace catalog.

Architecture 1: The Home Manager Registry Engine (Recommended for Global Config)

Instead of letting OMP execute imperative Git clones and network calls, your flake fetches plugin repositories as pinned inputs and generates OMP's internal installed_plugins.json (schema version 2) and marketplaces.json directly in the Nix store.

Flake Structure

```
{
  description = "Declarative OMP Marketplace & Plugin Config";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";

    # Upstream plugins pinned via Flake inputs
    plugin-context-mode = {
      url = "github:mariozechner/context-mode"; # example
      flake = false;
    };
    plugin-git-tools = {
      url = "github:my-org/omp-git-tools";
      flake = false;
    };
  };

  outputs = { self, nixpkgs, ... }@inputs:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forEachSystem = nixpkgs.lib.genAttrs systems;
    in {
      # Home Manager Module to include in your system/user configuration
      homeManagerModules.default = { config, lib, pkgs, ... }:
        let
          marketplaceName = "nix-declarative";

          # Define plugin specs
          declaredPlugins = {
            "context-mode" = {
              src = inputs.plugin-context-mode;
              version = "1.0.0";
            };
            "git-tools" = {
              src = inputs.plugin-git-tools;
              version = "0.1.0";
            };
          };

          # 1. Synthesize installed_plugins.json (Version 2 Schema)
          installedPluginsJson = pkgs.writeText "installed_plugins.json" (builtins.toJSON {
            version = 2;
            plugins = lib.mapAttrs' (name: cfg:
              lib.nameValuePair "${name}@${marketplaceName}" [
                {
                  scope = "user";
                  installPath = "${cfg.src}";
                  version = cfg.version;
                  installedAt = "1970-01-01T00:00:00.000Z";
                  lastUpdated = "1970-01-01T00:00:00.000Z";
                }
              ]
            ) declaredPlugins;
          });

          # 2. Synthesize marketplaces.json (Version 1 Schema)
          marketplacesJson = pkgs.writeText "marketplaces.json" (builtins.toJSON {
            version = 1;
            marketplaces = [
              {
                name = marketplaceName;
                sourceType = "local";
                sourceUri = "nix-store";
                catalogPath = ""; # Local injected registry bypasses catalog fetches
                lastUpdated = "1970-01-01T00:00:00.000Z";
                autoUpdate = "off";
              }
            ];
          });
        in {
          # Link files into OMP's default XDG data paths
          xdg.dataFile."omp/plugins/installed_plugins.json".source = installedPluginsJson;
          xdg.dataFile."omp/marketplaces.json".source = marketplacesJson;
        }
    }
  }
```

```

    # Prevent OMP from attempting background auto-updates over read-only store files
    xdg.configFile."omp/config.yml".text = ''
    marketplace:
      autoUpdate: "off"
    '';
  };
}

```

- **Pros:** Standard omp binary works unmodified; instant reproducibility across machines; plugin updates track `nix flake update`.
- **Caveat:** Because `installed_plugins.json` is a read-only symlink to `/nix/store`, interactive commands like `/marketplace install` in the TUI will throw `EACCES/EROFS` if they attempt to write.

Architecture 2: The Wrapped Binary (omp.withPlugins) for Per-Project Workspaces

OMP supports injecting plugin root directories via the `--plugin-dir` command-line flag. You can construct a wrapper derivation using `makeWrapper` that automatically launches OMP with all specified plugins injected into its runtime path.

This pattern is ideal for repo-specific devShells without mutating global state.

```

{
  description = "Wrapped OMP with isolated plugins";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    oh-my-pi.url = "github:can1357/oh-my-pi";

    # Plugin inputs
    plugin-sec-review = {
      url = "github:example/omp-sec-review";
      flake = false;
    };
  };

  outputs = { self, nixpkgs, oh-my-pi, ... }@inputs:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      ompPkg = oh-my-pi.packages.${system}.default;

      # Function to wrap OMP with arbitrary plugin inputs
      mkOmpWithPlugins = plugins:
        let
          pluginFlags = pkgs.lib.concatMapStringsSep " " (p: "--plugin-dir ${p}") plugins;
        in
          pkgs.symlinkJoin {
            name = "omp-wrapped";
            paths = [ ompPkg ];
            buildInputs = [ pkgs.makeWrapper ];
            postBuild = ''
              wrapProgram $out/bin/omp \
                --add-flags "${pluginFlags}"
            '';
          };
    in {
      packages.${system}.default = mkOmpWithPlugins [
        inputs.plugin-sec-review
      ];

      devShells.${system}.default = pkgs.mkShell {
        packages = [ self.packages.${system}.default ];
      };
    };
}

```

- **Pros:** Completely isolated per project; zero interaction with `$HOME` or global XDG dirs; no read-only file collisions with OMP's internal managers.
- **Caveat:** Plugins loaded purely via `--plugin-dir` bypass OMP's marketplace namespace tags (`@marketplace-name`).

Architecture 3: Building a Self-Contained Marketplace Derivation

If you want a private or team marketplace that adheres to OMP's catalog standard, build a derivation that generates `.omp-plugin/marketplace.json` and bundles plugin directories.

```

{
  description = "Declarative OMP Marketplace Catalog";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    code-review-plugin = {
      url = "github:example/code-review";
      flake = false;
    };
  };

  outputs = { self, nixpkgs, ... }@inputs:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};

      marketplaceCatalog = {

```

```

name = "team-marketplace";
description = "Internally curated plugins";
plugins = [
  {
    name = "code-review";
    description = "Automated diff analysis";
    version = "1.2.0";
    source = "./plugins/code-review";
  }
];
};

marketplacePackage = pkgs.runCommand "omp-marketplace" {} ''
  mkdir -p $out/.omp-plugin $out/plugins

  # Link plugin derivations
  ln -s ${inputs.code-review-plugin} $out/plugins/code-review

  # Generate catalog JSON
  cat <<'EOF' > $out/.omp-plugin/marketplace.json
  ${builtins.toJSON marketplaceCatalog}
  EOF
'';
in {
  packages.${system}.marketplace = marketplacePackage;
};
}

```

Once built, you can register it locally in OMP via:

```
omp /marketplace add $(nix build .#marketplace --print-out-paths --no-link)
```

Important OMP Constraints to Handle in Nix

1. Pure Skills vs. Bun Extensions:

- **Skills / Prompts / Agents / Hooks:** Plain Markdown/JSON (commands/, skills/, agents/) require no build step. A direct `inputs.foo` store path works out of the box.
- **Node/Bun Extensions (omp.extensions in package.json):** OMP attempts to create symlinks into `plugins/node_modules/` and updates `omp-plugins.lock.json`. If using TypeScript-based plugins, wrap or pre-bundle them with `bun2nix` or `buildNpmPackage` before referencing them in `installPath`.
- 2. **Auto-Updates:** Ensure `marketplace.autoUpdate`: "off" is set in `~/.config/omp/config.yml` or your OMP profile settings. Otherwise, OMP's background catalog freshness checks will complain when trying to reconcile versions against immutable Nix store paths.
- 3. **Scope Routing:** `installed_plugins.json` requires explicit "scope": "user" or "scope": "project". For Nix flakes managing system or home configurations, set scope to "user".

Sources

1. [oh-my-pi/docs/marketplace.md at main · GitHub](#)

User prompt: *i want it to be like extension to existing home-manager module, so when flake input added, i could do like programs.omp.plugins = ["github:great-user/nice-plugin", "marketplace:great-plugin"];*

Response: To extend Home Manager with a `programs.omp.plugins` option that accepts string specifiers like "github:user/repo" and "marketplace:plugin-name", you can implement a Home Manager module that parses these strings into derivations, resolves them against an upstream catalog, and writes OMP's internal `installed_plugins.json` and `marketplaces.json`.

1. The Flake Definition (flake.nix)

This flake exports `homeManagerModules.omp` (and default). It includes a default catalog input for the official marketplace and parser logic for string DSL entries.

```

{
  description = "Declarative Home Manager module extension for Oh My Pi (OMP)";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";

    # Pinned upstream marketplace catalog (for resolving "marketplace:*" plugins)
    omp-official-marketplace = {
      url = "github:can1357/oh-my-pi-marketplace";
      flake = false;
    };
  };

  outputs = { self, nixpkgs, omp-official-marketplace, ... }: {
    homeManagerModules.omp = { config, lib, pkgs, ... }:
      let
        cfg = config.programs.omp;

        # String parser and resolver
        resolvePlugin = p:
          if lib.isAttrs p then
            # Direct attrset support: { name, src, version ? "...", marketplace ? "..."}
            {
              name = p.name;
              marketplace = p.marketplace or "nix-declarative";
              version = p.version or "1.0.0";
              src = p.src;
            }
          else if lib.isString p then
            let
              ghMatch = builtins.match "github:([^\@]+)/([^\@]+)(@(.+))?" p;
              mpMatch = builtins.match "marketplace:(.+)" p;
            in

```

```

in
  if ghMatch ≠ null then
    let
      owner = builtins.elemAt ghMatch 0;
      repo = builtins.elemAt ghMatch 1;
      rev = let r = builtins.elemAt ghMatch 3; in if r ≠ null then r else "HEAD";
    in {
      name = repo;
      marketplace = "nix-declarative";
      version = rev;
      # Shallow fetch directly inside pure/impure eval
      src = builtins.fetchGit {
        url = "https://github.com/${owner}/${repo}.git";
        ref = if rev == "HEAD" then "HEAD" else "refs/heads/${rev}";
        allRefs = true;
      };
    }
  else if mpMatch ≠ null then
    let
      name = builtins.elemAt mpMatch 0;
      catalogFile = "${cfg.marketplaceCatalog}/omp-plugin/marketplace.json";
      catalog = builtins.fromJSON (builtins.readFile catalogFile);
      matched = lib.filter (x: x.name == name) catalog.plugins;
      entry =
        if matched ≠ []
        then lib.head matched
        else throw "OMP plugin '${name}' not found in the marketplace catalog!";
    in {
      inherit name;
      marketplace = catalog.name or "official";
      version = entry.version or "1.0.0";
      src =
        if lib.hasPrefix "http" entry.source
        then builtins.fetchGit { url = entry.source; }
        else "${cfg.marketplaceCatalog}/${entry.source}";
    }
  else
    throw "Unrecognized OMP plugin spec: '${p}'. Expected 'github:owner/repo[@ref]' or 'marketplace:name'."
else
  throw "Invalid type for OMP plugin: ${builtins.typeOf p}";

resolvedPlugins = map resolvePlugin cfg.plugins;

# 1. Synthesize installed_plugins.json (Schema v2)
installedPluginsJson = pkgs.writeText "installed_plugins.json" (builtins.toJSON {
  version = 2;
  plugins = lib.listToAttrs (map (p:
    lib.nameValuePair "${p.name}@${p.marketplace}" [
      {
        scope = "user";
        installPath = "${p.src}";
        version = p.version;
        installedAt = "1970-01-01T00:00:00.000Z";
        lastUpdated = "1970-01-01T00:00:00.000Z";
      }
    ]
  ) resolvedPlugins);
});

# 2. Synthesize marketplaces.json (Schema v1)
marketplacesJson = pkgs.writeText "marketplaces.json" (builtins.toJSON {
  version = 1;
  marketplaces = [
    {
      name = "nix-declarative";
      sourceType = "local";
      sourceUri = "nix-store";
      catalogPath = "";
      lastUpdated = "1970-01-01T00:00:00.000Z";
      autoUpdate = "off";
    }
    {
      name = "official";
      sourceType = "local";
      sourceUri = "${cfg.marketplaceCatalog}";
      catalogPath = ".omp-plugin/marketplace.json";
      lastUpdated = "1970-01-01T00:00:00.000Z";
      autoUpdate = "off";
    }
  ];
});
in {
  options.programs.omp = {
    enable = lib.mkEnableOption "Oh My Pi (OMP)";

    package = lib.mkPackageOption pkgs "omp" {
      default = null;
      nullable = true;
    };
  };
};

```

```

marketplaceCatalog = lib.mkOption {
  type = lib.types.path;
  default = omp-official-marketplace;
  description = "Source directory containing the .omp-plugin/marketplace.json catalog.";
};

plugins = lib.mkOption {
  type = with lib.types; listOf (either str (attrsOf anything));
  default = [];
  example = [
    "github:great-user/nice-plugin"
    "github:another-user/tool@v2.1.0"
    "marketplace:great-plugin"
  ];
  description = ''
    List of plugins specified as string URIs or attrsets:
    - 'github:owner/repo[@ref]'
    - 'marketplace:plugin-name'
    - '{ name = "foo"; src = inputs.foo; version = "1.0.0"; }'
  '';
};

config = lib.mkIf cfg.enable {
  home.packages = lib.optional (cfg.package ≠ null) cfg.package;

  # Link immutable states to XDG paths
  xdg.dataFile."omp/plugins/installed_plugins.json".source = installedPluginsJson;
  xdg.dataFile."omp/marketplaces.json".source = marketplacesJson;

  # Disable background updates to avoid collision with read-only store files
  xdg.configFile."omp/config.yml".text = ''
    marketplace:
      autoUpdate: "off"
  '';
};

homeManagerModules.default = self.homeManagerModules.omp;
};
}

```

2. Consuming the Module in Your Configuration

Add the module flake as an input and import it into your Home Manager configuration:

flake.nix (User System/Dotfiles Flake)

```

{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    home-manager = {
      url = "github:nix-community/home-manager";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  # The OMP extension flake
  omp-declarative = {
    url = "github:your-user/omp-declarative";
    inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { nixpkgs, home-manager, omp-declarative, ... }: {
    homeConfigurations."cyberrin" = home-manager.lib.homeManagerConfiguration {
      pkgs = nixpkgs.legacyPackages."x86_64-linux";
      modules = [
        omp-declarative.homeManagerModules.default
        ./home.nix
      ];
    };
  };
}

```

home.nix

```

{ pkgs, ... }: {
  programs.omp = {
    enable = true;

    plugins = [
      # Fetches default branch
      "github:great-user/nice-plugin"

      # Pins specific branch or tag
      "github:can1357/omp-git-tools@v1.2.0"

      # Resolves against the official marketplace catalog
      "marketplace:great-plugin"
    ];
}

```

```
    };
  }
}
```

Implementation Caveats

- **Flake Purity (builtins.fetchGit):** Calling builtins.fetchGit without an explicit SHA-1 commit hash or within an unpinned repository requires evaluating with --impure when rebuilding via home-manager switch. If strict flake evaluation (--pure) is required, pass flake inputs directly as attrsets in plugins:

```
plugins = [
  { name = "nice-plugin"; src = inputs.nice-plugin; version = "1.0.0"; }
  "marketplace:great-plugin"
];
```

- **Catalog Resolution:** The "marketplace:*" syntax reads .omp-plugin/marketplace.json at Nix evaluation time via builtins.readFile. If the upstream marketplace contains non-git tarball URLs, ensure the module handles them or override programs.omp.marketplaceCatalog with a local/pinned source tree.

User prompt: what if just use the version tagged "latest"

Response: Relying on a floating "latest" tag in a Nix flake breaks across four distinct layers: Git mechanics, Nix evaluator purity, store caching, and OMP's internal version engine.

1. Git Has No Built-In Latest Ref

Unlike Docker (image:latest) or npm (pkg@latest), Git does not have a native "latest" pointer.

- Git only understands branches (refs/heads/*), tags (refs/tags/*), and HEAD.
- When you pass ref = "latest" to Git or builtins.fetchGit, Git literally queries refs/tags/latest or refs/heads/latest.
- Unless the repository maintainer manually creates and pushes a tag explicitly named latest, the fetch fails immediately with:

fatal: couldn't find remote ref refs/tags/latest

GitHub's "Latest Release" is an HTTP REST API abstraction (/repos/:owner/:repo/releases/latest), not a native Git ref.

2. Pure Evaluation Fails in Nix Flakes

By default, flakes enforce pure evaluation mode.

If your module resolves "latest" or "HEAD" dynamically via builtins.fetchGit:

```
# This FAILS in pure mode
builtins.fetchGit {
  url = "https://github.com/user/plugin.git";
  ref = "HEAD"; # or "latest"
}
```

Nix will abort with:

error: in pure evaluation mode, 'fetchGit' requires a 'rev' attribute

To bypass this, you would have to run home-manager switch --impure every time. Even then, Nix caches git trees based on tarball-ttl (typically 1 hour or more). You will not reliably pull upstream updates when they release unless you manually invalidate the local eval cache.

3. Floating Tags Destroy Reproducibility and Generations

If an upstream repository does maintain a moving latest Git tag:

- **Generations Break:** If an upstream push introduces a bug and you roll back to yesterday's Home Manager generation, rebuilding or garbage-collecting will re-resolve the moving tag to the broken commit rather than your working state.
- **Fixed-Output Derivation Mismatches:** If you use pkgs.fetchFromGitHub (which requires a hash attribute), the moment upstream moves the latest tag, your build crashes with a hash mismatch error.

4. OMP Expects SemVer in installed_plugins.json

OMP's registry engine parses version fields for version checks and marketplace sync:

```
{
  "name@marketplace": [
    {
      "scope": "user",
      "installPath": "/nix/store/...",
      "version": "latest"
    }
  ]
}
```

OMP's marketplace commands use SemVer comparisons (semver.gt, semver.satisfies). Feeding the literal string "latest" instead of a standard x.y.z format causes invalid version warnings or silent upgrade check failures during TUI marketplace discovery.

The Nix-Idiomatic Alternatives

If the goal is "I don't want to track release numbers manually, I just want the current version", choose one of two approaches:

Option A: Delegate "Latest" to flake.lock (Recommended)

Let Nix resolve upstream HEAD once, lock the commit SHA, and update it on your schedule:

```
# flake.nix
inputs = {
  # Omit ref/tag: Nix automatically locks to the upstream default branch tip
  plugin-foo.url = "github:user/plugin-foo";
  plugin-foo.flake = false;
};
```

```

outputs = { self, ... }@inputs: {
  homeConfigurations.user = home-manager.lib.homeManagerConfiguration {
    modules = [{
      programs.omp.plugins = [
        {
          name = "plugin-foo";
          src = inputs.plugin-foo;
          version = inputs.plugin-foo.shortRev or "rolling";
        }
      ];
    }
  ];
};
};
};
};

```

Whenever you want to fetch upstream updates:

```
nix flake update plugin-foo
```

This stays 100% pure, deterministic, and rollback-safe while requiring zero manual version tracking.

Option B: External Tracking (npins or nvfetcher)

If you want automated tracking of GitHub's actual **latest release** (not just the `main` branch tip) without maintaining Flake inputs, use `npins` or `nvfetcher`.

They query GitHub's release API, resolve the latest tag, pin the commit and SHA256 in a generated JSON file, and let your Nix module consume the resulting output cleanly in pure evaluation mode.

Sources

1. [builtins.fetchGit - Nix function reference - Noogle](#)

User prompt: the thing is that i want to give it to the other users if they like it, so it is more for them, like no one wants to add version

Response: If you want end users to just drop strings into `programs.omp.plugins` without looking up commit SHAs, tags, or version numbers, you run directly into Nix's core constraint: **pure flake evaluation forbids non-deterministic network discovery**.

To give other users a frictionless experience where they never have to type a version, you have three practical patterns.

Pattern 1: Automatic Catalog Resolution (Zero-Version Marketplace)

If a user specifies a plugin from the marketplace (`"marketplace:plugin-name"`), **they should never need to write a version**.

Your flake pins the upstream OMP marketplace repository as an internal input. Your module reads its catalog, extracts the latest version and source path, and satisfies OMP's requirements automatically.

How it looks for the user:

```

# The user's home.nix - no versions, no inputs, no hashes
programs.omp.plugins = [
  "marketplace:context-mode"
  "marketplace:git-tools"
];

```

How your module resolves it:

```

# Inside your flake's module definition
let
  catalogFile = "${ompMarketplaceInput}/.omp-plugin/marketplace.json";
  catalog = builtins.fromJSON (builtins.readFile catalogFile);

  resolveMarketplacePlugin = name:
    let
      matches = builtins.filter (p: p.name == name) catalog.plugins;
      entry = if matches != [] then builtins.head matches
              else throw "OMP plugin '${name}' not found in official catalog!";
    in {
      inherit (entry) name;
      marketplace = catalog.name or "official";
      # Pull the version directly from the marketplace's own metadata
      version = entry.version or "1.0.0";
      # If source is relative in the repo, point to the store path
      src = if lib.hasPrefix "http" entry.source
            then builtins.fetchGit { url = entry.source; }
            else "${ompMarketplaceInput}/${entry.source}";
    };

```

When your module updates via `nix flake update`, all marketplace plugins bump automatically for your users.

Pattern 2: The Generated Lockfile (omp-Lock CLI)

For arbitrary GitHub URLs (`"github:owner/repo"`), users hate finding git revs and SHA-256 hashes manually. Tools like `spicetify-nix` and `nvfetcher` solve this by generating a companion lockfile.

Provide a tiny script in your flake:

```
nix run github:your-repo/omp-flake#lock
```

The User Flow:

1. The user lists the repositories in `home.nix` without versions:

```

programs.omp.plugins = [
  "github:mariozschner/context-mode"
  "github:user/awesome-plugin"
];

```

2. The user runs `nix run github:your-repo/omp-flake#lock` (or a helper alias).
3. The script queries GitHub's API for the default branch HEAD and generates `~/ .config/omp/plugins-lock.json`:

```
{
  "mariozechner/context-mode": {
    "rev": "8a3b5c1...",
    "hash": "sha256-abc...",
    "version": "1.2.0"
  }
}
```

4. Your module consumes the lockfile in pure mode:

```
lockData = builtins.fromJSON (builtins.readFile (config.xdg.configHome + "/omp/plugins-lock.json"));

resolveGitHubPlugin = slug:
  let
    lock = lockData.${slug} or (throw "Plugin '${slug}' not locked. Run 'omp-lock' first.");
    parts = lib.splitString "/" slug;
    repo = builtins.elemAt parts 1;
  in {
    name = repo;
    marketplace = "github";
    version = lock.version;
    src = pkgs.fetchFromGitHub {
      owner = builtins.elemAt parts 0;
      repo = repo;
      inherit (lock) rev hash;
    };
  };
};
```

Users never manage versions or hashes by hand, and their evaluations remain 100% pure and reproducible.

Pattern 3: The `pkgs.ompPlugins` Overlay (The Neovim / Spicetify Model)

This is the standard approach used by `vimPlugins`, `tmuxPlugins`, and `nur`. Instead of parsing raw strings at evaluation time, your flake packages a curated package set updated by a daily CI bot.

The User Flow:

```
# home.nix
programs.omp.plugins = with pkgs.ompPlugins; [
  context-mode
  git-tools
  awesome-plugin
];
```

How you implement it:

1. In your repository, maintain an `nvfetcher.toml` or simple JSON list of popular plugin URLs.
2. A GitHub Action runs `nvfetcher` weekly, committing updated commit hashes and SemVer tags to your repo.
3. Your flake exports an overlay:

```
overlays.default = final: prev: {
  ompPlugins = lib.mapAttrs (name: spec:
    pkgs.stdenv.mkDerivation {
      pname = name;
      inherit (spec) version src;
      installPhase = "cp -r . $out";
      passthru = {
        ompVersion = spec.version;
        ompMarketplace = "nix-curated";
      };
    }
  ) (import ./generated-plugins.nix { inherit (final) fetchgit; });
};
```

Users get IDE autocompletion for plugin names, zero version maintenance, and instant cache hits.

Pattern 4: The `--impure` Quick Fallback

If a user refuses to run a lock script or use pre-packaged sets, and insists on arbitrary `"github:owner/repo"` strings right now:

```
resolveImpure = p:
  let
    parts = builtins.match "github:([^\s]+)/([^\s]+)" p;
    owner = builtins.elemAt parts 0;
    repo = builtins.elemAt parts 1;
    src = builtins.fetchGit {
      url = "https://github.com/${owner}/${repo}.git";
      # Pulls default branch tip automatically
    };
  in {
    name = repo;
    marketplace = "impure-github";
    # Synthesize SemVer from commit date/rev so OMP does not crash
    version = "0.0.0-${builtins.substring 0 7 src.rev}";
    inherit src;
  };
};
```

- **Advantage:** Users just run `home-manager switch --impure` and everything resolves without touching versions.

- **Trade-off:** Evaluation requires network access and cannot run inside strict CI or pure flakes without `--impure`.

Which One to Ship?

Combine **Pattern 1** and **Pattern 4** with an optional **Pattern 2** lockfile:

1. For "marketplace:foo", resolve it automatically from the pinned catalog (**Pattern 1**). No version needed.
2. For "github:owner/repo", check if a local lock entry exists. If yes, use the locked commit in pure mode. If not, use `builtins.fetchGit` (requiring `--impure`) and synthesize the version automatically from `src.rev`.

This gives users zero-friction onboarding while leaving room for pure, reproducible setups.

User prompt: *ok what do i need for my develop flake to start hee job*

Response: A standard directory layout and starter `flake.nix` with an isolated evaluation test check will let you iterate on the module without touching your actual Home Manager configuration.

Project Structure

```
omp-nix/
├── flake.nix
├── flake.lock
├── modules/
│   └── omp.nix
├── tests/
│   └── test-config.nix
```

1. flake.nix

This flake exports the Home Manager module, sets up a developer shell with linting/JSON tooling, and provides a standalone test package (`nix build .#testConfig`) so you can inspect the synthesized JSON files instantly.

```
{
  description = "Declarative Oh My Pi (OMP) plugin manager for Home Manager";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    home-manager = {
      url = "github:nix-community/home-manager";
      inputs.nixpkgs.follows = "nixpkgs";
    };

    # Pinned upstream marketplace repository
    omp-official-marketplace = {
      url = "github:can1357/oh-my-pi-marketplace";
      flake = false;
    };
  };

  outputs = { self, nixpkgs, home-manager, omp-official-marketplace, ... }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forEachSystem = nixpkgs.lib.genAttrs systems;
    in {
      # 1. The Home Manager Module export
      homeManagerModules.omp = import ./modules/omp.nix {
        inherit omp-official-marketplace;
      };
      homeManagerModules.default = self.homeManagerModules.omp;

      # 2. Developer Shell
      devShells = forEachSystem (system:
        let
          pkgs = nixpkgs.legacyPackages.${system};
        in {
          default = pkgs.mkShell {
            packages = with pkgs; [
              # Formatting & Nix LSP
              alejandra
              statix
              nixd

              # Inspection & CLI debugging
              jq
              git
            ];

            shellHook = ''
              echo "OMP Declarative Module Dev Environment"
              echo "Run 'nix build .#test-json --impure' to preview generated JSON output."
            '';
          };
        }
      );

      # 3. Test derivation to verify generated JSON output without touching ~/.config
      packages = forEachSystem (system:
        let
          pkgs = nixpkgs.legacyPackages.${system};
```

```

# Minimal Home Manager evaluation
eval = home-manager.lib.homeManagerConfiguration {
  inherit pkgs;
  modules = [
    self.homeManagerModules.default
    ./tests/test-config.nix
  ];
};
in {
  # Outputs a directory containing the synthesized JSON files
  test-json = pkgs.runCommand "omp-test-json-output" {} ''
    mkdir -p $out
    cp ${eval.config.xdg.dataFile."omp/plugins/installed_plugins.json".source} $out/installed_plugins.json
    cp ${eval.config.xdg.dataFile."omp/marketplaces.json".source} $out/marketplaces.json
  '';
}
);
};
}

```

2. modules/omp.nix

The module handles string parsing, catalog reading, fallback resolution, and file generation.

```

{ omp-official-marketplace }:
{ config, lib, pkgs, ... }:

let
  cfg = config.programs.omp;

  # Parser: converts input types (attrs or strings) into normalized plugin records
  resolvePlugin = p:
    if lib.isAttrs p then {
      name = p.name;
      marketplace = p.marketplace or "nix-declarative";
      version = p.version or "1.0.0";
      src = p.src;
    }
    else if lib.isString p then
      let
        ghMatch = builtins.match "github:([^\@]+)/([^\@]+)(@(.+))?" p;
        mpMatch = builtins.match "marketplace:(.*)" p;
      in
        # Match 'marketplace:<name>'
        if mpMatch != null then
          let
            name = builtins.elemAt mpMatch 0;
            catalogPath = "${cfg.marketplaceCatalog}/.omp-plugin/marketplace.json";
            catalog = builtins.fromJSON (builtins.readFile catalogPath);
            matched = lib.filter (x: x.name == name) catalog.plugins;
            entry =
              if matched != []
              then lib.head matched
              else throw "OMP plugin '${name}' was not found in the catalog!";
          in {
            inherit name;
            marketplace = catalog.name or "official";
            version = entry.version or "1.0.0";
            src =
              if lib.hasPrefix "http" entry.source
              then builtins.fetchGit { url = entry.source; }
              else "${cfg.marketplaceCatalog}/${entry.source}";
          }

        # Match 'github:<owner>/<repo>[@<ref>]'
        else if ghMatch != null then
          let
            owner = builtins.elemAt ghMatch 0;
            repo = builtins.elemAt ghMatch 1;
            ref = let r = builtins.elemAt ghMatch 3; in if r != null then r else "HEAD";
            src = builtins.fetchGit {
              url = "https://github.com/${owner}/${repo}.git";
              ref = if ref == "HEAD" then "HEAD" else "refs/heads/${ref}";
              allRefs = true;
            };
          in {
            name = repo;
            marketplace = "github";
            version = "0.0.0-${builtins.substring 0 7 src.rev}";
            inherit src;
          }
        else
          throw "Invalid OMP plugin format: '${p}'. Expected 'github:owner/repo[@ref]' or 'marketplace:name'."
      else
        throw "Invalid type for programs.omp.plugins item: ${builtins.typeOf p}";

  resolvedPlugins = map resolvePlugin cfg.plugins;

  installedPluginsJson = pkgs.writeText "installed_plugins.json" (builtins.toJSON {

```

```

version = 2;
plugins = lib.listToAttrs (map (p:
  lib.nameValuePair "${p.name}@${p.marketplace}" [
    {
      scope = "user";
      installPath = "${p.src}";
      version = p.version;
      installedAt = "1970-01-01T00:00:00.000Z";
      lastUpdated = "1970-01-01T00:00:00.000Z";
    }
  ]
) resolvedPlugins);
});

marketplacesJson = pkgs.writeText "marketplaces.json" (builtins.toJSON {
  version = 1;
  marketplaces = [
    {
      name = "official";
      sourceType = "local";
      sourceUri = "${cfg.marketplaceCatalog}";
      catalogPath = ".omp-plugin/marketplace.json";
      lastUpdated = "1970-01-01T00:00:00.000Z";
      autoUpdate = "off";
    }
    {
      name = "github";
      sourceType = "local";
      sourceUri = "nix-store";
      catalogPath = "";
      lastUpdated = "1970-01-01T00:00:00.000Z";
      autoUpdate = "off";
    }
  ];
});
in {
  options.programs.omp = {
    enable = lib.mkEnableOption "Oh My Pi declarative plugin management";

    marketplaceCatalog = lib.mkOption {
      type = lib.types.path;
      default = omp-official-marketplace;
      description = "Path to the marketplace repository containing .omp-plugin/marketplace.json";
    };

    plugins = lib.mkOption {
      type = with lib.types; listOf (either str (attrsOf anything));
      default = [];
      example = [
        "marketplace:context-mode"
        "github:can1357/omp-git-tools"
      ];
      description = "List of plugins as strings or attrsets.";
    };
  };

  config = lib.mkIf cfg.enable {
    xdg.dataFile."omp/plugins/installed_plugins.json".source = installedPluginsJson;
    xdg.dataFile."omp/marketplaces.json".source = marketplacesJson;

    xdg.configFile."omp/config.yml".text = ''
      marketplace:
        autoUpdate: "off"
    '';
  };
}

```

3. tests/test-config.nix

A stub configuration file to verify evaluation during development.

```

{ pkgs, ... }: {
  # Home Manager requirements for standalone evaluation
  home.username = "testuser";
  home.homeDirectory = "/home/testuser";
  home.stateVersion = "24.05";

  programs.omp = {
    enable = true;
    plugins = [
      # Official marketplace test entry
      # (Replace with an existing name present in upstream .omp-plugin/marketplace.json)
      "marketplace:git-tools"

      # GitHub string test entry
      "github:mariozchner/context-mode"
    ];
  };
}

```

Quick Start Workflow

Initialize git, enter the shell, and build the test output:

```
git init
git add .
nix develop
nix build .#test-json --impure
jq . result/installed_plugins.json
```

The resulting `result/installed_plugins.json` will show the resolved Nix store paths, simulated SemVer revisions, and marketplace namespaces without writing anything to `$HOME`.

User prompt: omp does not search a central store or certify marketplace publishers. /marketplace discover searches only catalogs you have already added. Obtain a source from the plugin publisher's own site or repository, then inspect it before adding it: Confirm the repository or domain belongs to the expected publisher. Inspect .omp-plugin/marketplace.json or .claude-plugin/marketplace.json. Follow each plugin's source; a trusted catalog can still point to another repository. Prefer entries pinned to a reviewed commit with sha, rather than a moving branch. Adding a source fetches and validates its catalog; it does not install a plugin. List configured sources at any time to catch an unexpected or renamed source: /marketplace list From a shell: omp plugin marketplace list Accepted source forms are: SourceExampleHow omp treats itGitHub shorthandowner/repoClones the GitHub repositoryGit URLhttps://github.com/org/repo, git@github.com:org/repo.git, or ssh://...Clones the repositoryDirect catalog URLhttps://plugins.example.com/marketplace.jsonDownloads that JSON catalogLocal directory./marketplace, ~/marketplace, or an absolute pathReads the directory without cloning itGit and local sources must contain .omp-plugin/marketplace.json (preferred) or the Claude Code-compatible .claude-plugin/marketplace.json fallback. A direct catalog URL cannot use relative plugin sources such as ../plugins/linter, because there is no accompanying repository tree.

Response: Because OMP follows the decentralized, Claude Code-compatible plugin specification, there is no central registry, no npm-like package namespace, and no certified repository list.

Every plugin or collection of plugins is exposed through a catalog (`.omp-plugin/marketplace.json` or `.claude-plugin/marketplace.json`), and OMP expects users to:

1. Register a marketplace catalog (`marketplaces.json`).
2. Install a plugin from that catalog (`installed_plugins.json`).

Many plugin authors structure their repository as a **self-contained unit**: the repository is both the plugin and a single-entry marketplace catalog pointing to itself (often with `source: ". " or ./`). Other repositories act as **multi-plugin catalogs** (containing dozens of plugins in subdirectories or pointing to external repos).

This fits Nix cleanly. Because Nix resolves repositories to local `/nix/store` paths, we can treat every upstream source as a local catalog, eliminating network calls and auto-update issues in OMP.

The Architecture: Two-Tier Declarative Model

To give users the zero-config experience where they just write strings without tracking versions, the module should support two patterns:

1. **Direct Plugin Shorthand ("github:owner/repo")**: The module fetches the repo, auto-detects `.omp-plugin/marketplace.json` or `.claude-plugin/marketplace.json`, extracts the plugin metadata, and automatically registers both the catalog and the installed plugin.
2. **Multi-Plugin Catalog (marketplaces + plugins)**: The user declares a shared catalog once in `programs.omp.marketplaces`, then picks individual plugins in `programs.omp.plugins` by name ("plugin-name" or "plugin-name@marketplace-name").

Implementation: modules/omp.nix

```
{ config, lib, pkgs, ... }:

let
  cfg = config.programs.omp;

  # Helper: Read catalog JSON supporting both OMP and Claude Code fallbacks
  readCatalog = srcPath:
    let
      ompFile = "${srcPath}/.omp-plugin/marketplace.json";
      claudeFile = "${srcPath}/.claude-plugin/marketplace.json";
    in
      if builtins.pathExists ompFile then {
        path = ".omp-plugin/marketplace.json";
        data = builtins.fromJSON (builtins.readFile ompFile);
      }
      else if builtins.pathExists claudeFile then {
        path = ".claude-plugin/marketplace.json";
        data = builtins.fromJSON (builtins.readFile claudeFile);
      }
      else
        throw "No valid marketplace catalog found in ${toString srcPath}. Expected .omp-plugin/marketplace.json or .claude-plugin/marketplace.json";

  # Helper: Resolve string/path sources into a fetched Nix path
  fetchSource = src:
    if lib.isAttrs src || lib.isPath src then
      src
    else if lib.isString src then
      let
        ghMatch = builtins.match "github:([^\@]+)/([^\@]+)(@(.+))?" src;
        shorthandMatch = builtins.match "([^\@]+)/([^\@]+)(@(.+))?" src;
      in
        if ghMatch != null || shorthandMatch != null then
          let
            match = if ghMatch != null then ghMatch else shorthandMatch;
            owner = builtins.elemAt match 0;
            repo = builtins.elemAt match 1;
            ref = let r = builtins.elemAt match 3; in if r != null then r else "HEAD";
          in
            builtins.fetchGit {
              url = "https://github.com/${owner}/${repo}.git";
              ref = if ref == "HEAD" then "HEAD" else "refs/heads/${ref}";
              allRefs = true;
            }
        else if lib.hasPrefix "/" src || lib.hasPrefix "." src then
          /. + src
        else
          throw "Unsupported marketplace source: '${src}'"
```

```

else
  throw "Invalid type for source: ${builtins.typeOf src}";

# 1. Normalize registered marketplaces
# Produces: [ { name, sourceUri, catalogPath, catalogData } ]
configuredMarketplaces = map (entry:
  let
    rawSource = if lib.isAttrs entry then entry.src else entry;
    storePath = fetchSource rawSource;
    catalog = readCatalog storePath;
    marketplaceName = if lib.isAttrs entry && entry ? name
                      then entry.name
                      else catalog.data.name or (builtins.baseNameOf storePath);
  in {
    name = marketplaceName;
    sourceUri = "${storePath}";
    catalogPath = catalog.path;
    catalogData = catalog.data;
  }
) cfg.marketplaces;

# 2. Resolve installed plugins
# Handles direct "github:owner/repo" auto-registration AND "plugin[@marketplace]" lookups
resolvePlugin = pluginSpec:
  if lib.isAttrs pluginSpec then
    # Explicit attrset override
    {
      name = pluginSpec.name;
      marketplace = pluginSpec.marketplace or "custom";
      version = pluginSpec.version or "1.0.0";
      installPath = "${pluginSpec.src}";
      marketplaceEntry = null;
    }
  else if lib.isString pluginSpec then
    let
      isDirectRepo = builtins.match "github:.[^/]+/[^@]+" pluginSpec != null;
    in
      if isDirectRepo then
        # CASE A: User gave a repository directly.
        # We treat it as an ad-hoc marketplace containing its own plugin(s).
        let
          storePath = fetchSource pluginSpec;
          catalog = readCatalog storePath;
          catalogData = catalog.data;
          mktName = catalogData.name or (builtins.baseNameOf storePath);

          # If catalog lists plugins, use the first one; otherwise fallback to repo name
          firstPlugin =
            if (catalogData ? plugins) && (builtins.length catalogData.plugins > 0)
            then builtins.head catalogData.plugins
            else {
              name = mktName;
              version = "1.0.0";
              source = ".";
            };

          pluginInstallPath =
            if firstPlugin.source == "." || firstPlugin.source == "/" then
              "${storePath}"
            else
              "${storePath}/${firstPlugin.source}";
        in {
          inherit (firstPlugin) name;
          marketplace = mktName;
          version = firstPlugin.version or "1.0.0";
          installPath = pluginInstallPath;

          # Register as an ad-hoc catalog so OMP's /marketplace list sees it
          marketplaceEntry = {
            name = mktName;
            sourceUri = "${storePath}";
            catalogPath = catalog.path;
          };
        }
      else
        # CASE B: User specified "plugin-name" or "plugin-name@marketplace"
        let
          parts = lib.splitString "@" pluginSpec;
          targetName = builtins.elemAt parts 0;
          targetMkt = if builtins.length parts > 1 then builtins.elemAt parts 1 else null;

          # Search across all configured marketplaces
          findInMarketplaces = lib.filter (m:
            (targetMkt == null || m.name == targetMkt) &&
            (lib.any (p: p.name == targetName) (m.catalogData.plugins or []))
          ) configuredMarketplaces;

          matchedMkt =
            if findInMarketplaces != [] then builtins.head findInMarketplaces
            else throw "Plugin '${pluginSpec}' not found in any declared marketplaces.";

```

```

    pluginDef = builtins.head (lib.filter (p: p.name == targetName) matchedMkt.catalogData.plugins);

    resolvedPath =
      if lib.hasPrefix "http" pluginDef.source then
        "${builtins.fetchGit { url = pluginDef.source; }}"
      else if pluginDef.source == "." || pluginDef.source == "/" then
        matchedMkt.sourceUri
      else
        "${matchedMkt.sourceUri}/${pluginDef.source}";
    in {
      name = targetName;
      marketplace = matchedMkt.name;
      version = pluginDef.version or "1.0.0";
      installPath = resolvedPath;
      marketplaceEntry = null;
    }
  else
    throw "Invalid plugin specification: ${builtins.typeOf pluginSpec}";

resolvedPlugins = map resolvePlugin cfg.plugins;

# Merge explicitly configured marketplaces with ad-hoc ones from direct plugins
allMarketplaces =
  configuredMarketplaces ++
  (lib.filter (m: m != null) (map (p: p.marketplaceEntry) resolvedPlugins));

# Synthesize installed_plugins.json (Schema v2)
installedPluginsJson = pkgs.writeText "installed_plugins.json" (builtins.toJSON {
  version = 2;
  plugins = lib.listToAttrs (map (p:
    lib.nameValuePair "${p.name}@${p.marketplace}" [
      {
        scope = "user";
        installPath = p.installPath;
        version = p.version;
        installedAt = "1970-01-01T00:00:00.000Z";
        lastUpdated = "1970-01-01T00:00:00.000Z";
      }
    ]
  ) resolvedPlugins);
});

# Synthesize marketplaces.json (Schema v1)
# OMP reads local store directories cleanly via sourceType = "local"
marketplacesJson = pkgs.writeText "marketplaces.json" (builtins.toJSON {
  version = 1;
  marketplaces = map (m: {
    name = m.name;
    sourceType = "local";
    sourceUri = m.sourceUri;
    catalogPath = m.catalogPath;
    lastUpdated = "1970-01-01T00:00:00.000Z";
    autoUpdate = "off";
  }) (lib.unique allMarketplaces);
});

in {
  options.programs.omp = {
    enable = lib.mkEnableOption "Oh My Pi (OMP) declarative plugin management";

    marketplaces = lib.mkOption {
      type = with lib.types; listOf (either str (attrsOf anything));
      default = [];
      example = [
        "github:anthropics/claude-code-plugins"
        "./local-marketplace"
      ];
      description = ''
        Catalogs containing .omp-plugin/marketplace.json or .claude-plugin/marketplace.json.
        Can be GitHub shorthand ('owner/repo'), local paths, or store derivations.
      '';
    };

    plugins = lib.mkOption {
      type = with lib.types; listOf (either str (attrsOf anything));
      default = [];
      example = [
        # Direct repository (auto-registers catalog + plugin)
        "github:mariozchner/context-mode"

        # Named plugin from an entry in `programs.omp.marketplaces`
        "pr-review"
        "code-runner@claude-code-plugins"
      ];
      description = ''
        Plugins to install. Accepts:
        - Direct GitHub shorthand ('owner/repo')
        - Plugin names defined in one of the configured 'marketplaces'
        - 'plugin-name@marketplace-name' to disambiguate
      '';
    };
  };
}

```

```

    '';
  };
};

config = lib.mkIf cfg.enable {
  xdg.dataFile."omp/plugins/installed_plugins.json".source = installedPluginsJson;
  xdg.dataFile."omp/marketplaces.json".source = marketplacesJson;

  xdg.configFile."omp/config.yml".text = ''
    marketplace:
      autoUpdate: "off"
  '';
};
}

```

How Users Consume This

Scenario 1: Standalone Plugins (Zero-Friction)

If a user just wants standalone plugins from GitHub, they don't even need to touch marketplaces:

```

programs.omp = {
  enable = true;

  plugins = [
    # Automatically clones, reads .omp-plugin/marketplace.json,
    # pins the version from the repo's catalog, and registers both.
    "github:mariozechner/context-mode"
    "github:can1357/omp-git-tools"
  ];
};

```

Scenario 2: Multi-Plugin Marketplace

If a team or community publishes a catalog containing multiple tools:

```

programs.omp = {
  enable = true;

  # 1. Register the catalog
  marketplaces = [
    "github:my-org/curated-omp-catalog"
    "~/code/my-private-plugins # local directory works directly!
  ];

  # 2. Pick only what you want from those catalogs
  plugins = [
    "linter"
    "security-audit"
    "my-dev-tool@my-private-plugins"
  ];
};

```

What This Accomplishes

- **Strict Catalog Compliance:** Automatically supports both `.omp-plugin/marketplace.json` and the Claude Code `.claude-plugin/marketplace.json` standard.
- **No Version Tracking for Users:** Metadata and versions are parsed straight from the publisher's catalog file.
- **Zero OMP Background Corruption:** Setting `sourceType = "local"` with `autoUpdate = "off"` informs OMP that the catalogs and plugins live locally in the filesystem, preventing background network sync attempts against read-only `/nix/store` paths.
- **Inspectable via CLI:** Inside OMP, running `/marketplace list` or `omp plugin marketplace list` accurately lists the registered sources pointing cleanly to their immutable store paths.